Reviewer Pack — Minimal Frobenius-Schur Index and Boolean Circuit Width Corr…

Entry 1cc41088410f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 16:37:36 UTC

Minimal Frobenius-Schur Index and Boolean Circuit Width Correlation

Entry ID: `1cc41088410f` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-08 16:37:36 UTC

1. Conjecture

Field A (mathematical branch): Algebra (Frobenius-Schur Indicators) **Field B** (complexity object): Boolean Circuits (Circuit Width)

Statement:

> For any given boolean circuit C with n inputs and width w(C), the Frobenius-Schur index of its normal form, denoted as FS_index(C), is linearly correlated with the circuit width w(C), such that FS_index(C) = Θ(w(C)).

Rationale (proposer's reasoning):

> The Frobenius-Schur indicator provides a measure of noncommutativity in an algebraic structure and could potentially expose structural complexity in boolean circuits. If this correlation holds, it suggests that certain measures of noncommutativity are closely tied to the complexity of computation.

Taxonomy category: `FrobeniusSchur` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `a39c6185e7e96a81`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the Pearson correlation coefficient between Frobenius-Schur index (FS_index(C)) and circuit width (w(C)) for at least 80% of the seeds exceeds 0.7, with an average absolute difference between FS_index(C) and Θ(w(C)) across all seeds not exceeding 3.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.5s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_circuit(n):
        circuit = []
        for _ in range(n):
            gate = random.choice(['AND', 'OR'])
            inputs = [random.randint(0, 1) for _ in range(random.randint(2, n))]
            circuit.append((gate, inputs))
        return circuit

    def calculate_frobenius_schur_index(circuit):
        # Placeholder implementation of Frobenius-Schur index calculation
        # This is a dummy function and should be replaced with actual logic
        return random.random()

    def calculate_circuit_width(circuit):
        width = 0
        for gate, inputs in circuit:
            width = max(width, len(inputs))
        return width

    n_values = [5, 10, 15, 20, 30, 40]
    fs_index_sum = 0
    width_sum = 0
    instances_tested = 0
    n_max = 0

    for n in n_values:
        for _ in range(5):  # Sample 5 circuits per size
            circuit = generate_random_circuit(n)
            fs_index = calculate_frobenius_schur_index(circuit)
            width = calculate_circuit_width(circuit)
```

```
            if fs_index is None or width is None:
                return {
                    "metric_name": "FS_index vs Width",
                    "metric_value": 0,
                    "instances_tested": instances_tested,
                    "n_max": n_max,
                    "conjecture_holds": False,
                    "counterexample": "mapping_undefined"
                }

            fs_index_sum += fs_index
            width_sum += width
            instances_tested += 1
            n_max = max(n_max, n)

    mean_fs_index = fs_index_sum / instances_tested
    mean_width = width_sum / instances_tested

    correlation_coefficient = (instances_tested * sum(fs_index * width for fs_index, width in zip(
                               mean_fs_index * instances_tested - mean_width * instances_tested) /
                              math.sqrt((instances_tested * sum(fs_index**2 for fs_index in range(
                                        (instances_tested * sum(width**2 for width in range(instan

    if correlation_coefficient > 0.7 and abs(mean_fs_index - mean_width) <= 3:
        return {
            "metric_name": "FS_index vs Width",
            "metric_value": correlation_coefficient,
            "instances_tested": instances_tested,
            "n_max": n_max,
            "conjecture_holds": True,
            "counterexample": ""
        }
    else:
        return {
            "metric_name": "FS_index vs Width",
            "metric_value": correlation_coefficient,
            "instances_tested": instances_tested,
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": "correlation_not_met"
        }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value)**2 for result in
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
```

```
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample='correlation_not_met' first_failing_seed={first_f
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
': 40, 'conjecture_holds': False, 'counterexample': 'correlation_not_met'}
TRIAL: {'metric_name': 'FS_index vs Width', 'metric_value': 0.998403925505927, 'instances_tested': 3
TRIAL: {'metric_name': 'FS_index vs Width', 'metric_value': 0.9983996844333747, 'instances_tested': 
TRIAL: {'metric_name': 'FS_index vs Width', 'metric_value': 0.9983955035623868, 'instances_tested': 
TRIAL: {'metric_name': 'FS_index vs Width', 'metric_value': 0.9983953076899369, 'instances_tested': 
TRIAL: {'metric_name': 'FS_index vs Width', 'metric_value': 0.9983908204542952, 'instances_tested': 
TRIAL: {'metric_name': 'FS_index vs Width', 'metric_value': 0.998402107275279, 'instances_tested': 3
TRIAL: {'metric_name': 'FS_index vs Width', 'metric_value': 0.9983999995332871, 'instances_tested': 
TRIAL: {'metric_name': 'FS_index vs Width', 'metric_value': 0.9984012876178244, 'instances_tested': 
TRIAL: {'metric_name': 'FS_index vs Width', 'metric_value': 0.9984004933357878, 'instances_tested': 
TRIAL: {'metric_name': 'FS_index vs Width', 'metric_value': 0.9984082455638199, 'instances_tested': 
RESULT: FALSIFIED counterexample='correlation_not_met' first_failing_seed=11
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> The test code uses a placeholder implementation for the Frobenius-Schur index calculation, which is not faithful to the mathematical definition of the index. The correlation coefficient and mean values are computed based on a random output from this placeholder function, making the results unreliable.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Safety rail: critic_challenge_falsified | original: The test results indicate that the conjecture does not hold for at least one seed, as the Pearson correlation coefficient was below the required thres | next: Investigate the calculation of the Frobenius-Schur index to ensure it is accurate and faithful to the mathematical definition. Additionally, retest the conjecture with a correct implementation to verify its validity.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12360
2	preregistration	ollama_remote	glm4:latest	0	0	17969
3	novelty	ollama_remote	glm4:latest	0	0	8652

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	novelty	ollama_remote	glm4:latest	0	0	8944
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20114
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11192
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10010
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	32254
9	critic	ollama_remote	glm4:latest	0	0	50554
10	judge	ollama_remote	glm4:latest	0	0	11274

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 183323 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/1cc41088410f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1cc41088410f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/1cc41088410f.tar.gz` (if generated)